

AI System Building with GenAI as a Cognitive Partner

Group Members:

Joshua Fuquen

Chris Tham

Annie Wang

i) What We Learned About GenAI in This Project

During this final project, we learned that GenAI is useful not only for getting answers, but also for improving the way we think about problems. At first, it was easy to treat GenAI like a tool that could simply fix the code for us. However, the purpose of this project was different. We had to use GenAI as a cognitive partner, meaning that it helped us reflect, question our assumptions, and understand the reason behind each bug. This made the debugging process more meaningful because we were not just copying solutions. We were learning how to identify the actual cause of an error.

One important lesson was that a weak reflection usually focuses on the wrong part of the problem. For example, in one case, the program failed because the number of labels did not match the number of text examples. A weak reflection might say that the model was bad or that LogisticRegression could not understand the data. However, GenAI guided us to think about the relationship between input examples and output labels. This helped us realize that every text example needs one matching label. The real issue was not the model itself, but the structure of the dataset.

Another example was when the vectorizer was changed from `fit_transform` to only `transform`. At first, this looked like a small change, but it caused an important problem because the vectorizer had not learned the vocabulary from the training data. GenAI did not directly give the solution. Instead, it asked us to think about what the vectorizer needs before it can convert text into numbers. This helped us understand that some tools in machine learning have a required order of steps. The vectorizer must first learn the vocabulary, and only after that can it transform text into numerical features.

We also learned that GenAI can help us separate superficial reasoning from deeper reasoning. A superficial explanation usually says something general, such as “the dataset is too small” or “the model is confused.” A deeper explanation explains exactly what changed, how that change affected the system, and why the error happened. Through the “Good” and “Bad” labels, GenAI helped us evaluate the quality of our own thinking. This was useful because debugging is not only about fixing code. It is also about learning how to explain problems clearly and logically.

Overall, this project showed us that GenAI is most helpful when it is used to support human reasoning, not replace it. If we ask GenAI for the direct answer, we may fix the bug quickly, but we do not learn as much. When GenAI gives Socratic guidance, it pushes us to slow down, inspect the code carefully, and connect the bug to the specific change we made. That process helped us become more confident in understanding our AI system.

ii) What We Learned About Building an AI System in General

This project also helped us understand that building an AI system is more than choosing a model and running training code. Even a simple classification system has many connected parts, including the dataset, labels, preprocessing, model training, prediction, and evaluation. If one part is changed incorrectly, the entire system can fail or behave differently than expected.

One major lesson was the importance of data quality and structure. The model depends on the dataset being organized correctly. If the number of labels does not match the number of text examples, the system cannot train properly. If the labels are incorrect or inconsistent, the model may learn the wrong patterns. This showed us that data preparation is just as important as the model itself. A machine learning system can only learn from the information it receives.

We also learned that preprocessing is a critical step in AI systems. In our text classification project, the model could not understand raw sentences directly. The text first had to be converted into numerical features using `TfidfVectorizer`. When we skipped this step or used the wrong data format during training or prediction, the system failed. This helped us understand that AI models usually require data to be transformed into a format they can process.

Another important lesson was that training and prediction must use consistent formats. If the model is trained with vectorized data, it must also receive vectorized data when making predictions. Passing raw text into a model trained on numerical vectors creates errors because the model expects the same type of input it saw during training. This taught us that consistency across the pipeline is very important.

We also learned that evaluation must be done carefully. Accuracy should compare the true labels with the predicted labels. If we accidentally compare the input text with the predictions, the result is incorrect. This showed us that evaluation metrics are only meaningful when they are used with the right data. A model score can be misleading if the evaluation step is not set up correctly.

In general, this project helped us see an AI system as a full pipeline, not just a model. The dataset, preprocessing, training, prediction, and evaluation all depend on each other. Small changes can create unexpected problems, especially when we do not understand the role of each component. By intentionally creating bugs, we learned how fragile an AI system can be and how important it is to debug carefully.

In conclusion, we learned that GenAI can be a powerful learning partner when it helps us reflect instead of simply giving us answers. We also learned that building AI systems requires careful attention to data, preprocessing, model training, and evaluation. This project helped us improve both our technical understanding and our ability to reason through problems step by step.